How cost-efficient is scaling up versus scaling out Valkey for different read/write workloads?

Eryk Kściuczyk
Technische Universität Berlin
Berlin, Germany
e.ksciuczyk@campus.tu-berlin.de

***Abstract*—The process of scientific writing is often tangled up with the intricacies of typesetting, leading to frustration and wasted time for researchers. In this paper, we introduce Typst, a new typesetting system designed specifically for scientific writing. Typst untangles the typesetting process, allowing researchers to compose papers faster. In a series of experiments we demonstrate that Typst offers several advantages, including faster document creation, simplified syntax, and increased ease-of-use.**

***Index Terms*—benchmark, cloud, Valkey, vertical scalability, horizontal scalability, cost**

I. Introduction

A. Motivation

B. Research Question

II. Background

A. Benchmarking Concepts

B. System Under Test (SUT)

III. Benchmark Design

A. Benchmark Objectives

The benchmark goals are to measure throughput, latency and success-rate in multiple harware configurations. We chose those metrics due to being relevant for performance of a database. We chose success-rate in order to measure whether the increase amount of RAM will lead to more cache-hits under semi-realistic load.

We want to compare which of two scaling strategies, horizontal or vertical, is more efficient. In other words, how much extra throughput, latency and success-rate does a dollar give us.

B. Use Case and Workload

We generate load on a system by simulating synthetic semi-realistic cache database traffic that we generate using our custom data-generator. The traffic simulates only two Redis standardized requests, SET and GET. The operations are performed on keys sampled from a Zipf Distribution (exponent:1.1, offset:1.0) as it resembles realistic traffic, some keys are retrieved much more often than some other ones [hot spots]. We perform a micro-benchmark as all the key operations are independent.

We use seeded random data generator so that we have consistency between runs, thus ensuring reproducibility and helping with repeatability. The generated keys are of length 24, 7 characters for a prefix and 16 for the key. We use 7 characters for a prefix, as often usecase of Redis compatible databases is to add a table name in the prefix e.g. "users" and we chose 16 for the key as it is the length of a UUID, which is commonly used. This way we make the results of the benchmark more realistic.

The workload of the benchmark is dynamic and it starts with 10% GET requests and 90% SET and linearily shifts to 90% and 10% SET through the benchmark run.

We generate the load using a separate `t2d-standard-48` VM running in GCP[LINK HERE] in the same region and zone as SUT using our custom writen toow. We use the same size of the load-generator VM for all the benchmark runs and configurations. During all runs we have monitored the resource utilization of the load-generator to ensure it is not the bottleneck. The load-generator uses a fixed pool of 256 workers, each having it's own connection to the DB and sending another request as soon as it receives a response from the previous one until the benchmark finishes.

C. System Architecture

The architecture of our benchmark is straightforward, we have a load-generator node and an SUT node/nodes (depends on scalability strategy). The load-generator sends requests and saves the results to a local CSV file, which is retrieved after a benchmark run to be processed offline afterwards. After each benchmark run the whole infrastructure is removed, to ensure one run doesn't influence the other one by keeping some data in cache.

The requests are sent directly to the appropriate SUT nodes as the client libraries used are cluster aware, thus they cache the key ranges that belong to each node, removing the need for a layer 7 load-balancer.

To ensure reproducibility we use infrastrucutre as code using `terraform` and bash scripts for setup. This way others may reproduce our benchmark by using the same machine types, network configuration, and setup, configuration of Valkey.

We use Google Cloud Platform (GCP) due to free credits provided by the university as benchmarking large VM sizes and clusters quickly gets expensive. We run the experiments in `us-west1` region in zone `c`, due to high default total core count quote for the `t2d` CPU type we chose. For the load-

generator we use `t2d-standard-48` to match the biggest benchmarked VM type for vertical scalability and matching the total core count of the largest vertically scaled cluster.

All the nodes were deployed on the same cloud provider in the same region and zone to reduce the network communication costs and potential variability in connection.

In the table Table I we show the different VM setups along with their prices and the category they compete in. We defined four price categories for setups. Each cateogory represents a price range:

1) Budget-entry-level ~14$/month
2) Entry-level ~50$/month
3) Mid-range ~200$/month
4) High-performance ~666$/month

The Budget-entry-level price range contains only a single node as in this price range there were no options to form a cluster, as it was the cheapest VM of `t2d-standard` available. We included this category to see what performance one can achieve on limited budget, as the Valkey performance is still highly single threaded.

TABLE I
TABLE SHOWING ALL CATEGORIES AND THE VM SETUPS CHOSEN FOR THEM

VM Type	CPU Cores	Memory	Price
Budget-entry-level			
1x t2d-standard-1	1	4GB	$13.88
Entry-level			
1x t2d-standard-4	4	16GB	$55.52
3x t2d-standard-1	3x1	12GB	$41.64
Mid-range			
1x t2d-standard-16	16	64GB	$222.06
7x t2d-standard-2	7x2	56GB	$194.32
High-performance			
1x t2d-standard-48	48	192GB	$666.18
24x t2d-standard-2	24x2	192GB	$666.24
48x t2d-standard-1	48x1	192GB	$666.24

D. Benchmark Implementation

We use custom benchmark tool implemented in Go by us, available at GitHub[1]. Go programming language has been chosen to great performance because of it being compiled language and great support for concurrency, making the code easier to understand for potential reviewers and increasing the velocity of the development.

To improve repeatability we used `mise`, to control the versions of tools alongside standard dependency tracking of Go. By tools we mean gcloud, google-cloud-pricing-cost-calculator go, terraform and uv

Each hardware configuration has been benchmarked 3 times to try to mitigate performance variability of the cloud provider. We run each Benchmark 20 minutes long, the duration is limited but was chosen due to limited available GCP credits.

We don't give Valkey a warm-up time and start measuring the results since beginning, the same applies for the shutdown time. To prevent unreliable data, we discard the first 5 minutes of benchmark and last 2 minutes of benchmark run in offline analysis. The minutes at the end are discarded as some Goroutines can already start quitting and thus making the results not representative.

E. Experiment Variables

a) *Independent Variables:*
b) *Dependent Variables:*

We measured latency by saving calculated difference between sent request and response. Additionally we save time when the request was sent, the request type (GET or SET). Afterwards using this saved data we calculate throughput.

F. Measurement Methodology

Data has been collected between 30.01.2026-04.02.2026. We collected the CSV results files from load-generator using rsync. We observed the utilization of CPU cores, Memory, Network, Disk manually using `btop` during benchmark runs.

IV. RESULTS

In this section we present our results by showing a table per one price range. This way we compare the horizontally scaled setups with their price equivalent verticallly scaled setups.

A. Summary

First we show the table Table II with the summary of the results, and then we delve deeper into individual metrics.

TABLE II
SYSTEM PERFORMANCE AND COST EFFICIENCY ACROSS DEFINED PRICE CATEGORIES.

Type	**Node Count**	**Throughput (RPS)**	**Latency p50 (ms)**	**Latency p99 (ms)**	**Latency p99.9 (ms)**	**Efficiency (RPS/$)**
Budget Entry Level						
Single	1x 1cores	46488 ± 1671	4.01 ± 0.27	15.51 ± 0.87	21.68 ± 2.04	1507.41
Entry Level						
Single	1x 4cores	114801 ± 4993	2.14 ± 0.11	3.86 ± 0.15	9.58 ± 0.48	930.63
Cluster	3x 1cores	105270 ± 7063	1.08 ± 0.04	10.46 ± 1.45	15.11 ± 1.22	1137.82
Mid Range						
Single	1x 16cores	202438 ± 13911	1.19 ± 0.07	2.43 ± 0.37	6.09 ± 0.34	410.26
Cluster	7x 2cores	241606 ± 10513	1.04 ± 0.04	2.08 ± 0.18	14.66 ± 14.21	559.59
High Performance						
Single	1x 48cores	192225 ± 21297	1.22 ± 0.11	3.76 ± 0.64	7.91 ± 0.85	129.86
Cluster	24x 2cores	200404 ± 22836	1.24 ± 0.10	2.54 ± 0.90	9.61 ± 9.64	135.38
Cluster	48x 1cores	177921 ± 4921	1.41 ± 0.04	2.41 ± 0.05	4.71 ± 0.04	120.19

[1]https://github.com/erykksc/valkey-benchmark

In table Table II we see that in Entry-level category, the single VM setup achieves higher throughput and lower p99 and p99.9 latencies with lower standard deviation compared to Cluster setup. On the other hand, cluster setup achieves almost two times lower latency in p50 with better price efficiency.

The situation changes in the Mid-range where the cluster achieves better throughput with smaller standard deviation, lower latencies in p50 and p99 and better efficiency. It is worth mentioning that we observed high standard deviation of 14.21ms in the p99.9 latency in the cluster setup which is 41.79 times higher than the one of Single node setup. Additionally there is improvement of almost 5 fold in the p99 latency of the clustered setup in mid-range compared to entry-level clustered setup.

In High-performance category, we observe that we have reached scalability limits of both strategies. Both Single node and Clustered setup performed worse than their respective setups in Mid-range. Furthermore, in this category we benchmarked two Cluster setups one with 24 VMs with 2 cores and 48 VMs with 1 core each, where the setup with lower amount of VMs proved to get higher throughput but with high variability of 22836 RPS. The setup of 48 VMs with 1 core showed low latencies (below 0.05ms) in p50, p99 and p99.9 but with lower throughput than the 7 VMs with 2 cores from the mid-range category.

B. *Cost Efficiency*

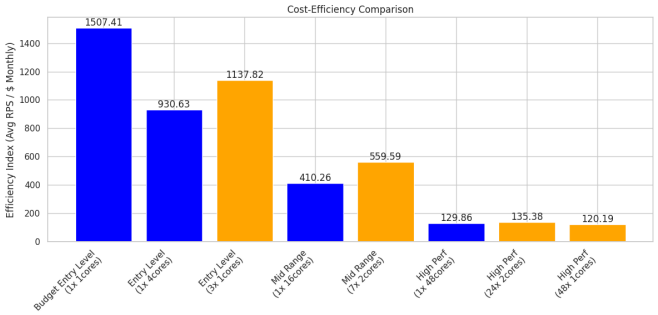

Fig. 1. Cost Efficiency Comparison plot for all VM setups benchmarked

In Fig. 1 plot we can observe a trend that the vertically scaled setups achieve higher efficiencies than their comparable horizontally scaled ones.

Additionally we observe high cost efficiency for the setup in Budget-entry-level showing high performance of a single CPU single VM setup.

Another trend in the results is that the more resources we use the lower the efficiency, both for vertically and horizontally scaled systems.

C. *Latencies*

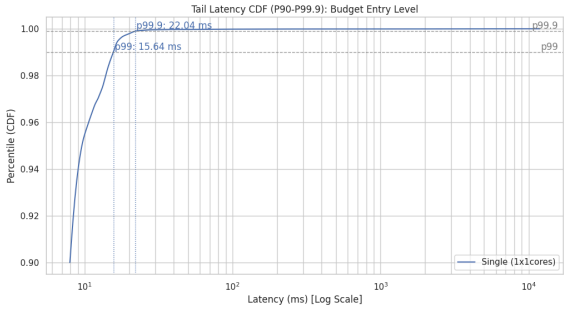

Fig. 2. Cumulative Distribution Function (CDF) of tail latencies in Budget-entry-level category

The Fig. 2 plot shows the cumulative distribution function (CDF) for budget entry level where we see a smooth almost vertical line reaching p99.9 at 21.68ms. We read the exact value from Table II.

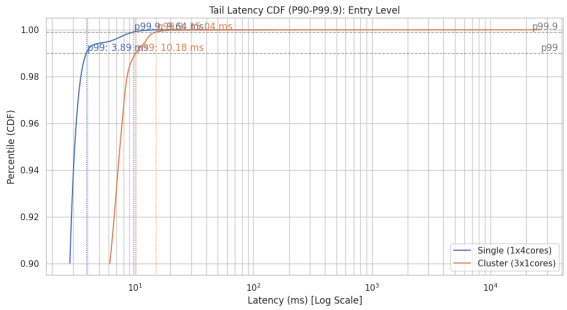

Fig. 3. Cumulative Distribution Function (CDF) of tail latencies in Entry level category

We compare the Entry-level setups in Fig. 3 and see that the tail latency CDF for Single VM setup is steeper than the Clustered one and reaches both p99 and p99.9 faster.

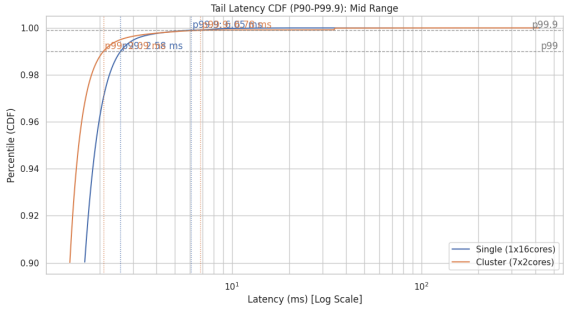

Fig. 4. Cumulative Distribution Function (CDF) of tail latencies in Mid-range category

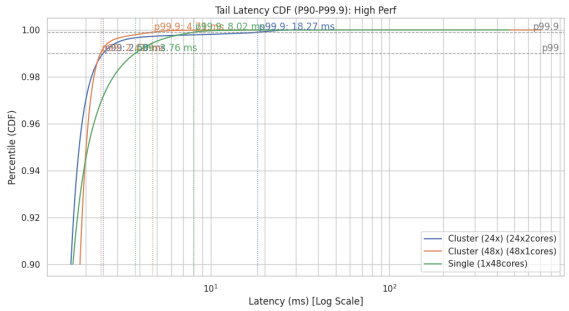

Fig. 5. Cumulative Distribution Function (CDF) of tail latencies in High-performance category

D. Workload Transition

We run the benchmark with changing load over time. The requests initially have 90% probability of being SET requests and 10% GET requests and this shift linearly with time to 10% SET and 90% GET requests. Thus we plot the throughput and p99 latency over time to see if there are some patterns throught time.

In Fig. 2 we see the workload transition plot for Budget-entry-level category, where

Fig. 6. Workload Transition Plot portraying throughput (RPS) and p99 latency (ms) for Budget-entry-level

In Fig. 3 we see the workload transition plot for Budget-entry-level category, where

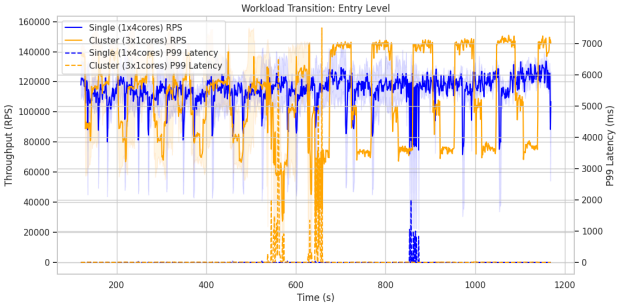

Fig. 7. Workload transition over time for the Entry-level setup, comparing single-node (blue) and cluster (orange) throughput and p99 latency. Solid lines show throughput, dashed lines show p99 latency, and the semi-transparent shaded regions in the background indicate variability in the measurements over time, highlighting the spread around the observed performance levels between benchmark runs.

In Fig. 7 we see that the cluster achieves higher peak throughput in Entry-level but suffers from significant instability. The single node delivers lower but much more stable throughput.

Both cluster and single VM setups contains periodic dips. In the second half of the experiment, where there are more GET requests, the clustered setup exhibits a clear periodic pattern where throughput alternates between two stable states: a prolonged high-throughput plateau and a prolonged low-throughput plateau. Transitions between these states occur swiftly, producing a square-wave–like oscillation in performance.

Additionally there are two clusters of latency spikes for the Clustered setup and one such cluster for the single VM setup. The latency has almost zero variability outside of those clusters.

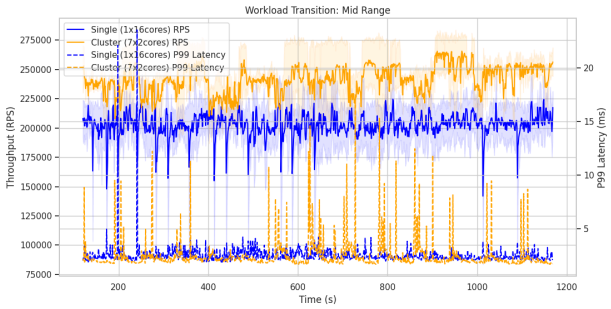

Fig. 8. Workload Transition Plot portraying throughput (RPS) and p99 latency (ms) for Mid-range

On the Fig. 8 we see that clustered setup achieved higher throughput, the single cluster setup appears more stable with ocasional dips. The latency plot for the single node setup is more consistent throughout the time with periodic small latency spikes, whereas clustered setup has lower latency but with more sporadic magnitude higher latency spikes compared to the single VM setup. The latency spikes in clustered setup appear mostly on the right side of the plot, so on during time when there are more GET requests than SET requests.

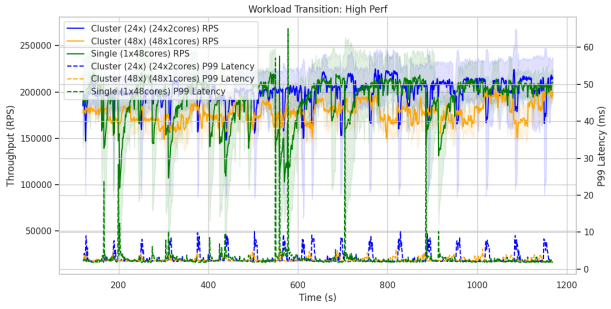

Fig. 9. Workload Transition Plot portraying throughput (RPS) and p99 latency (ms) for High-performance

On Fig. 9 we compare the High-performance setups throughout time and wee see more stable throughput in clus-

tered setups than in the single VM setup. The throughput in single VM setup shows steep dips in performance.

The latencies in two clustered setups include periodic spikes, the single VM setup has lower periodic spikes with occasional very high ones.

V. Evaluation/Discussion

VI. Threats to Validity

A. Internal Validity

- Measurement inaccuracies
- Benchmark implementation bias

B. External Validity

- Realism of workload

The workload in the experiment is semi-realistic as it uses Zipf distribution, but only simple commands SET and GET, benchmarking only the basic functionality of Valkey. Redis compatible databases such as Valkey, provide a wide set of commands which performant remains to be benchmarked.

- Cloud environment influence

The cloud environment introduces variability into the measurements, as the performance may vary between different times of a day as we can have a busy neighbor problem.

C. Construct Validity

- Metric suitability

VII. Conclusion

VIII. Future Work

This work could be extended by testing wider range of VM types to see how the scalability behaves in other sizes. One could also measure how the systems behave with more client connections, whether the vertical cluster can sustains higher simultaneous clients.

The length of the benchmark runs in this experiment were relatively short, and it would be preferable to run each of them for longer e.g., 1 hours instead of 20 minutes. This way one could obtain more reliable results, and observe the trends better.

One could also try to run the Valkey on a VM Type with 1 or 2 cores and a high amounts of memory, as the experiment results showed that single core performance gets great results.

IX. References

References